

Before starting, make sure you **read all the questions** and understand what is asked of you. Write your answers to the questions in a separate sheet of paper and make sure you **justify all of your answers**. Multiple-choice questions have a **unique correct answer**.

1. (10 points) Suppose we are given a **deterministic finite automaton** for a language $\mathcal{L}$ on Σ . For all words $w \in \Sigma^*$, we can use the automaton to determine whether $w \in \mathcal{L}$ in time:
 - A. $\mathcal{O}(2^{|w|})$
 - B. $\mathcal{O}(|w|^2)$
 - C. $\mathcal{O}(|w|)$
 - D. $\mathcal{O}(\log |w|)$
2. (10 points) Suppose we are given a **context free grammar** for a language $\mathcal{L}$ on Σ . For all words $w \in \Sigma^*$, we can use the grammar to determine whether $w \in \mathcal{L}$ in time:
 - A. polynomial in $|w|$
 - B. exponential in $|w|$
 - C. nope, that is an undecidable problem
3. (10 points) Give an example of an LL(1) grammar that is not **strongly LL(1)**.
4. (10 points) Give an example of an LL(k+1) language that is not LL(k). You may choose the value of k to suit your example.
5. (10 points) Give an example of an LR(0) language that is not LL(1).
6. (10 points) Give an example of a grammar and a word from its language which can be deterministically parsed by an LALR(1) parser thanks to ItemFollow and which cannot be done using Follow only.
7. (10 points) During the course we have focused on conservative approximations of both type checking and reachability analysis. Argue that the exact versions of both tasks are in fact **undecidable**.
8. (20 points) Let $T = \{\text{cst}, \text{id}, +, *\}$ be a set of terminals. Is the **language of the following grammar** SLR(0)? Is it LALR(2)?

(1)	Exp	$\rightarrow$	Exp + Prod
(2)		$\rightarrow$	Prod
(3)	Prod	$\rightarrow$	Prod * atom
(4)		$\rightarrow$	atom
(5)	atom	$\rightarrow$	cst
(6)		$\rightarrow$	id

9. (10 points) Give an arithmetic-expression tree such that the minimal number of registers required to evaluate it is exactly the Strahler number of the tree.